is used to specify additional behavior for the spec. Finally, describe expects a block, which contains the test suite itself.

As you'll see in a little bit, describe calls can be nested. By convention, the outermost call typically has the name of the class under test. The outermost describe call must be invoked as RSpec.describe, since that call starts outside of RSpec's control. Nested calls can use just plain describe, since RSpec manages those calls internally.

The actual spec is defined with the it method, which takes an optional string argument that documents the spec, an optional amount of metadata, and then a block that is the body of the spec. The string argument is not used internally to identify the spec—you can have multiple specs with the same description string. Again, the metadata is used to adjust RSpec's behavior.

RSpec also defines specify as an alias for it. Normally, you'd use it when the method takes a string argument to give the spec a readable natural-language name. (Historically the string argument started with "should," so the name would be something like "it should be valid," but that documentation pattern is no longer considered a good practice.) For single-line tests in which a string description is unnecessary, you can use specify to make the single line read more clearly, such as this:

```
specify { expect(user.name).to eq("fred") }
```

On line 6 you write your first testable expectation about the code:

```
expect(project.done?).to be_truthy
```

The general form of an RSpec expectation is expect(actual_value).to(matcher), with the parentheses around the matcher often omitted in practice. In this case, the expectation would more formally read, expect(project.done?).to(be_truthy). A *matcher* is an RSpec object that takes a value and determines if it matches expectations based on some set of logic. In this example, the matcher is be_truthy.

Let's trace through what RSpec does with this expectation. First is the expect call itself, expect(project.done?). RSpec defines the expect method, which takes in any object as an argument and returns a special RSpec proxy object called an ExpectationTarget.

The ExpectationTarget holds on to the object that was the argument to expect, and itself responds to two messages: to and not_to. (Okay, technically three messages, since to_not exists as an alias.) Both to and not_to are ordinary Ruby methods that expect a single argument, which needs to be an RSpec matcher.